Guía WISPR: retos variados y dificultad adaptativa

Oct 12, 2026 · @Tikvah

Resumen

Los retos se sienten repetitivos porque el sistema está armado para servir siempre el mismo contenido: la IA no se llama en ningún momento, los archivos son iguales para todos y la dificultad es solo una etiqueta que cambia los puntos.

Esta guía dice **qué archivo tocar, qué cambiar y cómo saber que quedó bien**. No trae el código hecho: el equipo lo escribe. Cada fase se puede trabajar en su propia rama `feature/<algo>` y entrar a `main` por Pull Request, como dice su `CLAUDE.md`.

Causas encontradas en el repositorio:

1. `backend/app/services/ai_advisor.py` existe, pero ninguna ruta ni servicio llama a `generate_challenge_spec_raw()`.
2. Los 8 retos Wi-Fi salen de `challenges/static/generar_wifi.js` con semilla fija (`wispr-static-v1`): mismo `.pcap` y misma flag para todos.
3. `network-wep-01` y `crypto-xor-01` leen plantillas con narrativa y flag escritas a mano, y su archivo se genera una sola vez al sembrar.
4. En `challenge_generator.py` las funciones ignoran `spec.difficulty`: un XOR fácil y uno difícil son idénticos.
5. En `services/scoring.py`, `nextChallenge` siempre devuelve `None`: no existe ninguna lógica de dificultad adaptativa.
6. El registro de técnicas solo tiene 2 entradas (`iv_reuse_wep`, `xor_singlebyte`).

Qué revela el challenges.json

El archivo tiene 581 retos, pero ninguno pasaría la validación de `ChallengeSpec` tal como está, y el lote generado por IA es más genérico que el lote hecho a mano. Usarlo como está (para servir retos o para entrenar el LoRA) va a producir justo lo que se quejan: retos parecidos entre sí.

Hallazgo	Dato	Por qué importa
Dos lotes de distinta calidad	180 del 9 sep (hechos a mano) y 401 del 2 oct (42 jobs de IA)	El lote de IA imita al primero y pierde detalle
Soluciones genéricas	88 de 401 retos de IA dicen "utiliza herramientas" sin decir cuáles; 0 en el lote manual	Un reto sin pasos concretos no se puede resolver ni generar
Narrativas repetidas	63 descripciones del lote IA empiezan "Un sistema de" y 57 "Un conjunto de"	Es la repetición que ven los jugadores
Mediana de la solución	146 caracteres (manual) contra 96 (IA)	Menos detalle técnico
Categorías incompatibles	`stego`, `reverse` y `misc` no existen en el schema; `network` y `osint` no aparecen en el JSON	Pydantic rechaza todo lo que no esté en el `Literal`
`technique` en texto libre	567 valores distintos	El schema exige uno de un registro fijo
Formato de flag viejo	Las 581 usan `FLAG{...}`	El schema y la terminal ya usan `WISPR{...}`
Flags repetidas	13 flags duplicadas, ej. `FLAG{traffic_analysis_reveals_secrets}` 3 veces	Retos distintos con la misma respuesta
Flags que revelan el tema	Ej. `FLAG{vigenere_is_weak}`	La flag debería ser imposible de adivinar
Títulos repetidos	41 duplicados, ej. "Cifrado de Datos" 6 veces	Se ven como el mismo reto
Mezcla de dificultad fija	Los 42 jobs pidieron siempre easy 3 / medium 4 / hard 3	La dificultad no responde al jugador
Filtro de calidad casi nulo	Ningún job rechazó más de 3 de 10	Entra casi todo lo que la IA produce

Hallazgo	Dato	Por qué importa
Registro con error	`web-wifi-66` trae un campo `!difficulty` además de `difficulty`	Dato sucio

Conclusión para el equipo: el lote del 9 de septiembre es la base buena. El del 2 de octubre debe revisarse a mano o descartarse antes de usarlo en cualquier lado.

Orden de trabajo y reglas

Trabajen las fases en este orden: las fases 1, 2 y 4 resuelven la queja principal sin depender de la API de Together, así que sirven para EXPOTEC aunque la IA no esté lista.

1. **Fase 1:** limpiar el banco de retos (datos).
2. **Fase 2:** dificultad real en los generadores.
3. **Fase 3:** archivo y flag únicos por jugador.
4. **Fase 4:** dificultad adaptativa.
5. **Fase 5:** IA solo para narrativa y pistas.
6. **Fase 6:** sumar técnicas nuevas, de a una.

Reglas que aplican a todas las fases:

- **La dificultad la decide el código, no la IA.** La IA escribe texto; los parámetros que hacen un reto más difícil los fija el generador.
- **La flag la genera el backend, nunca la IA.** Así no se puede adivinar ni repetir.
- **Nada pasa al generador sin validar con `ChallengeSpec`.** Esa regla ya existe; no la rompan.
- **Cada cambio lleva su prueba** en `backend/tests/unit/`. Si no hay prueba, el cambio no está terminado.
- **Actualicen `CLAUDE.md`** al cerrar cada fase, en la sección de checkpoint.

Fase 1: limpiar el banco de retos

Objetivo: que el `challenges.json` tenga la misma forma que `ChallengeSpec` y que solo queden retos concretos y resolubles.

Qué hacer:

- [ ] Decidir la lista final de categorías. O agregan `stego`, `reverse` y `misc` al `Literal` de `category` en `backend/app/schemas/challenge.py`, o los reclasifican. Elijan una opción y anoten por qué en `CLAUDE.md`.
- [ ] Cambiar todas las flags de `FLAG{...}` a `WISPR{...}`. En la fase 3 las flags pasan a generarse solas; por ahora solo se corrige el formato.
- [ ] Separar el texto libre de `technique` en dos campos: `technique` (un valor del registro, o vacío si todavía no hay generador) y `technique_notes` (la explicación en texto).
- [ ] Revisar a mano los 401 retos del 2 de octubre. Criterio para quedarse con uno: la solución nombra herramientas y pasos concretos, y el reto dice qué archivo recibe el jugador. Si no cumple, se borra.
- [ ] Eliminar flags duplicadas y renombrar títulos repetidos.
- [ ] Corregir `web-wifi-66` (borrar el campo `!difficulty`) y el título "etadata de Imagen" (`reverse-wifi-33`).
- [ ] Revisar el generador de jobs que produjo el lote: debería rechazar retos con solución genérica, flag repetida o título ya usado.

Cómo saber que quedó bien:

- [ ] Un script de prueba carga cada reto del JSON con `ChallengeSpec(**reto)` y ninguno falla.
- [ ] Una prueba verifica que no hay flags ni títulos repetidos.

Aviso sobre el LoRA: si este JSON es el dataset de ~500 ejemplos, entrenar con el lote de IA sin limpiar enseña al modelo a ser genérico. Limpien primero, entrenen después.

Fase 2: dificultad real en los generadores

Objetivo: que `easy`, `medium` y `hard` produzcan archivos distintos de verdad, no solo más puntos.

Archivo principal: `backend/app/services/challenge_generator.py`.

Qué hacer:

- [ ] Que cada función generadora lea `spec.difficulty` y cambie su comportamiento según el nivel.
- [ ] Definir una tabla de niveles por técnica, escrita en el código (no en el prompt). Ideas de partida:

Técnica	easy	medium	hard
`xor_singlebyte`	Llave de 1 byte	Llave de 2 a 4 bytes repetida	Llave de varios bytes y el resultado en base64 o hex
`iv_reuse_wep`	2 paquetes, plaintext documentado en la pista	El paquete conocido entre 20 a 50 paquetes de ruido	IV repetido escondido entre cientos de paquetes con IVs distintos, sin decir cuál es el conocido

- [ ] Quitar los valores fijos que hacen idéntico cada archivo: `_REUSED_IV`, `_AP_MAC`, `_STA_MAC` y las IPs del ARP conocido. Que se elijan al azar en cada generación.
- [ ] Si la tabla crece, pueden agregar a `ChallengeSpec` un campo opcional `params` que valide cada nivel; pero que los valores los ponga el backend, no la IA.

Cómo saber que quedó bien:

- [ ] En `tests/unit/test_challenge_generator.py`, una prueba por nivel que genere el archivo y lo resuelva con el método esperado (por ejemplo, fuerza bruta de 256 llaves solo funciona en `easy`).
- [ ] Una prueba que genere dos veces el mismo reto y confirme que los archivos son distintos.

Fase 3: archivo y flag únicos por jugador

Objetivo: que cada jugador reciba su propio archivo con su propia flag, para que no se la puedan pasar y para que repetir un reto no sea copiar la respuesta.

Archivos involucrados: `backend/app/api/challenges.py` (ruta `POST /challenges/active`), `backend/app/services/challenges.py`, y la tabla de instancias que ya menciona la migración `dba06aee3303`.

Qué hacer:

- [ ] Revisar qué tiene la tabla de instancias en esa migración. Necesita, como mínimo: jugador, reto, flag de esa instancia, ruta del archivo y fecha.

- [ ] En `POST /challenges/active`: si el jugador no tiene instancia de ese reto, generarla en ese momento con `materialize_challenge()`; si ya tiene, reutilizarla.
- [ ] Generar la flag en el backend con una parte aleatoria, por ejemplo `WISPR{xor_` + 8 caracteres hex + `}`, usando el módulo `secrets` de Python.
- [ ] Nombrar el archivo por instancia, no por id de reto (hoy es `{spec.id}.pcap`, y se sobrescribe).
- [ ] Cambiar `scoring.submit_flag()` para que compare contra la flag de la instancia del jugador, no contra la del catálogo.
- [ ] Cambiar `artifact_path()` para que solo sirva el archivo de la instancia del jugador que lo pide.
- [ ] Portar los 8 retos Wi-Fi de `generar_wifi.js` a Python para que también sean por jugador. Ya está anotado como pendiente en ese script (punto 7b del plan).

Cómo saber que quedó bien:

- [ ] Prueba de integración: dos jugadores activan el mismo reto y reciben flags distintas.
- [ ] Prueba: la flag del jugador A, enviada por el jugador B, se rechaza.
- [ ] Prueba: el jugador B no puede descargar el archivo del jugador A.

Fase 4: dificultad adaptativa

Objetivo: que al resolver un reto, el sistema recomiende el siguiente con una dificultad ajustada al desempeño del jugador. Usen reglas simples primero: son explicables ante el profesor y se pueden demostrar en vivo.

Archivos: `backend/app/services/scoring.py` (donde `nextChallenge` hoy siempre es `None`) y un servicio nuevo sugerido, `backend/app/services/difficulty.py`.

Qué hacer:

- [ ] Crear una función que calcule el nivel recomendado de un jugador a partir de `submissions` y `solves`: intentos fallidos, tiempo hasta resolver y si usó pistas.
- [ ] Definir las reglas por escrito antes de programarlas. Punto de partida para ajustar con datos:

Situación en el último reto	Siguiente nivel
Resolvió en el primer o segundo intento, sin pistas	Sube un nivel

Situación en el último reto	Siguiente nivel
Resolvió con 3 a 5 intentos o con una pista	Se mantiene
Más de 5 intentos fallidos, o lo abandonó	Baja un nivel
Ya está en el nivel más alto o más bajo	Se queda ahí

- [ ] Para registrar el uso de pistas, el frontend tiene que avisar cuando el jugador abre una. Revisen si ya existe esa llamada; si no, agréguenla.
- [ ] Llenar `nextChallenge` en la respuesta de `submit_flag()`: un reto no resuelto, de la misma categoría si hay, en el nivel calculado.
- [ ] Evitar repetir: no recomendar un reto que el jugador ya resolvió.
- [ ] Guardar el nivel actual por jugador y categoría (alguien puede ser bueno en crypto y estar empezando en forensics).

Cómo saber que quedó bien:

- [ ] Una prueba por cada fila de la tabla de reglas.
- [ ] Prueba: `nextChallenge` nunca devuelve un reto ya resuelto.

El `CLAUDE.md` menciona Scikit-learn para análisis. Pueden usarlo después, con datos reales de uso; con las reglas de arriba alcanza para el MVP.

Fase 5: conectar la IA para narrativa y pistas

Objetivo: que la IA aporte variedad en la historia de cada reto, sin decidir la flag, la técnica ni la dificultad.

Requisito previo: una `TOGETHER_API_KEY` funcionando en `backend/.env`. Según `CLAUDE.md` todavía está pendiente.

Archivo principal: `backend/app/services/ai_advisor.py`.

Qué hacer:

- [ ] Cambiar el prompt: que el backend le pase la categoría, la técnica y la dificultad ya decididas, y la IA devuelva solo `title`, `description` y `hint`.
- [ ] Pasarle un tema elegido al azar de una lista fija (ej. cafetería, hospital, colegio, fábrica, evento deportivo) para que no repita escenarios.
- [ ] Pasarle los títulos de los últimos retos que vio ese jugador, con la instrucción de no repetirlos.

- [ ] Llamarla desde `POST /challenges/active` al crear la instancia (fase 3), y guardar el texto en la instancia.
- [ ] Programar el respaldo que ya promete `docs/pipeline_ia_retos.md`: si la IA falla, tarda más de unos segundos, no hay API key o el JSON no valida, usar la plantilla fija. El jugador nunca debe ver un error por esto.
- [ ] Un límite de llamadas por jugador para no gastar el presupuesto de $300-400 USD del proyecto.

Cómo saber que quedó bien:

- [ ] Prueba con la IA simulada (mock) que devuelve JSON inválido: el reto se crea igual con la plantilla.
- [ ] Prueba sin API key: el reto se crea igual.
- [ ] Prueba: la flag de la instancia nunca sale del texto de la IA.

Fase 6: ampliar el registro de técnicas

Objetivo: pasar de 2 técnicas a varias, para que haya variedad real de mecánicas y no solo de historias. Sumen de a una, cada una con su prueba.

Para cada técnica nueva, los tres pasos que ya documenta `challenge_generator.py`:

1. Agregarla al `Literal` de `technique` en `backend/app/schemas/challenge.py`.
2. Escribir su función generadora, con sus niveles easy, medium y hard (fase 2).
3. Registrarla en `TECHNIQUE_REGISTRY` y escribir su prueba.

Candidatas que ya aparecen en el lote manual del JSON y se pueden generar con Python y Scapy:

Técnica sugerida	Categoría	Reto del JSON que la inspira
`caesar_channel`	crypto	`crypto-wifi-04` (ROT-N con N = canal)
`base64_layers`	crypto	`crypto-wifi-03` (base64 doble en un IE)
`xor_bssid`	crypto	`crypto-wifi-02` (llave = último octeto del BSSID)
`ssid_fragments`	misc	`misc-wifi-01` (flag partida en varios SSIDs)

Escojan primero las que reutilizan código que ya tienen (XOR, frames 802.11).

Checklist final y demo en EXPOTEC

El trabajo está listo cuando un mismo jugador puede jugar el mismo reto dos veces y ver un archivo, una flag y una historia distintos, y cuando el siguiente reto que recibe cambia de nivel según cómo le fue.

- [ ] Todas las pruebas de `backend/tests/` pasan con `pytest`.
- [ ] Todo el `challenges.json` valida con `ChallengeSpec`.
- [ ] Dos jugadores reciben flags distintas para el mismo reto.
- [ ] Un reto `hard` no se resuelve con el método del `easy`.
- [ ] `nextChallenge` sube o baja de nivel según la tabla de reglas.
- [ ] Si la IA falla o no hay API key, la plataforma sigue funcionando.
- [ ] `CLAUDE.md` actualizado con lo que se decidió en cada fase.

Guión sugerido para la demo:

1. Mostrar un reto fácil y resolverlo rápido, sin pistas.
2. Mostrar que el siguiente reto recomendado sube de nivel.
3. Fallar varias veces a propósito y mostrar que el siguiente baja.
4. Abrir el mismo reto con otra cuenta y comparar: archivo y flag diferentes.